

Project Report On “Dementia Detection”

Project Supervisor: Dr. S.K. Tripathi



Department of Computer Science & Engineering

NATIONAL INSTITUTE OF TECHNOLOGY PATNA

University Campus, Bihta - 801103

Submitted by:

Name	Roll No.
Omkar Kumar	2206010
Raju Kumar	2206012
Ravi Kumar	2206003

Course Code: CS64123

Table of Contents

1. Problem Statement
2. Motivation
3. Introduction
4. Methodology
 - 4.1. Data Collection
 - 4.2. Data Preprocessing
 - 4.3. Architecture
 - 4.3.1. CNN
 - 4.3.2. C-BAM
 - 4.3.3. Vision Transformer
 - 4.4. Implementation Details
5. Results
6. Conclusion
7. References

1. Problem Statement

- **Context:** Alzheimer's Disease is a progressive neurological disorder requiring early detection for effective management.
- **The Issue:**
 - Subtle structural changes in MRI scans (like "Very Mild" dementia) are difficult to distinguish from "Non Demented" brains using traditional methods.
 - Medical datasets often suffer from severe **class imbalance**, causing standard models to be biased toward the majority class (Healthy/Non-Demented) and fail on critical minority classes (Moderate Demented).

Objective: To build a robust Deep Learning model that handles class imbalance effectively and combines local feature extraction (CNN) with global context understanding (Transformers) to classify MRI scans into four stages of dementia.

2. Motivation

- **Clinical Need:** Automated, high-accuracy screening tools can assist radiologists in identifying early-stage dementia, reducing human error.
- **Technical Gap:** Traditional CNNs excel at edges and textures but struggle with long-range spatial dependencies (global context). Vision Transformers (ViT) handle global context but require massive data.
- **Our Solution:** A **Hybrid Architecture** combining the efficiency of CNNs with the global attention of Transformers, augmented by a custom data balancing pipeline to ensure fair diagnosis across all stages.

3. Introduction

Alzheimer's Disease (AD) is a progressive neurodegenerative disorder that irreversibly destroys memory and thinking skills, eventually disrupting the ability to carry out simple tasks. As the global population ages, the prevalence of AD is rising, creating an urgent need for effective, early diagnostic tools. Early diagnosis is critical because while there is currently no cure, early intervention can help manage symptoms and improve the quality of life for patients and their families.

Traditional diagnosis of Alzheimer's often relies on clinical assessments and manual interpretation of Magnetic Resonance Imaging (MRI) scans by radiologists. However, distinguishing between the early stages of cognitive impairment—specifically between "Non-Demented" (healthy) brains and those with "Very Mild" dementia—can be incredibly challenging due to the subtle and microscopic nature of the structural changes in the brain. This subjective interpretation can lead to variability in diagnosis.

Automated diagnostic systems using deep learning face significant technical hurdles. Medical datasets are notoriously imbalanced, often containing many more scans of healthy individuals than of those with severe conditions. Standard Convolutional Neural Networks (CNNs), while excellent at detecting local features like tissue texture and edges, often struggle to capture the long-range, global dependencies across the entire brain structure that are vital for holistic diagnosis.

This project addresses these challenges by proposing a novel **Hybrid CNN-ViT Architecture**. By fusing the local feature extraction capabilities of CNNs with the global context understanding of Vision Transformers (ViT), and augmenting this with Convolutional Block Attention Modules (CBAM), we aim to create a robust, high-accuracy classification model. This model is designed to not only distinguish between four distinct stages of dementia but also to handle the inherent class imbalance of medical data through a rigorous preprocessing pipeline.

4. Proposed Methodology

To overcome the limitations of using either CNNs or Transformers in isolation, this project creates a custom Hybrid Architecture that synergizes three powerful deep learning technologies:

1. Convolutional Neural Network (CNN): CNNs are the backbone of the model, utilized for their exceptional ability to extract local features. In the context of MRI, the CNN layers identify edges, textures, and boundaries between brain tissues (e.g., gray matter vs. white matter).
2. CBAM (Convolutional Block Attention Module): To enhance the CNN's feature extraction, CBAM is integrated into the convolutional blocks. It acts as an adaptive attention mechanism that refines features in two dimensions:
 - Channel Attention: Determines *what* features are important (e.g., prioritizing signals relevant to atrophy over noise).
 - Spatial Attention: Determines *where* in the image the important features are located, helping the model focus on relevant brain regions.
3. Vision Transformer (ViT): While CNNs focus on local details, they often miss global context. ViT addresses this by treating the image features as a sequence, using self-attention mechanisms to capture long-range dependencies. This allows the model to "connect the dots" between distant parts of the brain, understanding the global structural integrity characteristic of different dementia stages.

4.1. Dataset description

The foundation of this study is the Alzheimer's MRI 4-Class Dataset, a collection of neuroimaging data specifically curated for multi-class classification tasks.

- **Data Type:** The dataset consists of 2D Magnetic Resonance Imaging (MRI) scans provided in grayscale. Grayscale is utilized to eliminate color artifacts and focus the model's learning on structural density and tissue integrity, which are the primary indicators of atrophy.
- **Resolution:** All images have been preprocessed and resized to a standard resolution of 176 $\times$ 208 pixels to ensure consistency across the input layer of the neural network.
- **Classification Categories:** The dataset is categorized into four distinct classes representing the progression of the disease:
 1. **Non Demented:** Scans of healthy control subjects showing no signs of dementia.
 2. **Very Mild Demented:** Scans showing the earliest, often subtle, signs of cognitive impairment.
 3. **Mild Demented:** Scans indicating clear but early-stage dementia progression.
 4. **Moderate Demented:** Scans representing severe cognitive decline. This is the minority class in the dataset, making it the most difficult for standard models to learn without bias.
- **Validation Strategy:** To ensure a rigorous and honest evaluation of the model's real-world performance, 20% of the data is held out as a validation set. Crucially, this validation split preserves the original, unbalanced distribution of classes found in the real world, preventing misleadingly high accuracy scores that can occur with artificially balanced test sets.

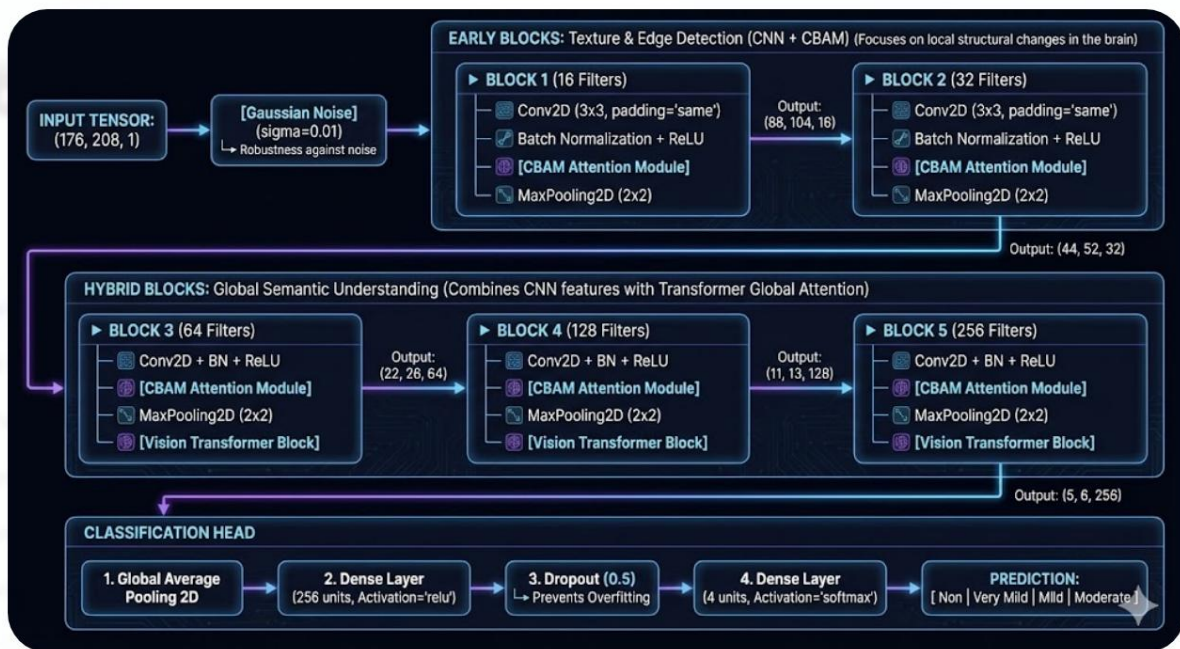
4.2. Data Preprocessing Pipeline

Data preparation is a critical phase in medical image analysis, directly impacting model performance and generalization.

- **Image Loading:** Images are loaded using OpenCV in grayscale mode. This dimensionality reduction (from 3 RGB channels to 1 channel) reduces computational complexity and focuses the model on the relevant structural features of the brain tissue.
- **Stratified Split:** The dataset is split into training (80%) and validation (20%) sets using a stratified approach. This ensures that the class distribution (the ratio of Non-Demented to Moderate Demented, etc.) remains consistent between the training and validation sets, preventing any single class from being entirely absent in the evaluation phase.
- **Handling Class Imbalance (Oversampling):** A major challenge with this dataset is the severe imbalance, particularly the scarcity of "Moderate Demented" examples. To prevent the model from becoming biased toward the majority class ("Non Demented"), we apply **Oversampling** to the training set.
 - **Target:** The minority classes are synthetically expanded so that all four classes in the training set reach a balanced count of approximately **2560 images** each. This forces the model to treat all stages of dementia with equal importance during learning.
- **Data Augmentation:** To prevent overfitting—where the model memorizes specific training images rather than learning general features—an image data generator is applied to the oversampled data. This creates variations of the images on the fly using:
 - **Rotation:** Randomly rotating images by up to 8 degrees.

- **Zoom:** Applying a random zoom range of 5%.
- **Shift:** Randomly shifting the image width and height by 5%. These augmentations simulate slight variations in patient positioning during scanning, making the model more robust

4.3 Architecture design



The model is constructed in a hierarchical manner, progressively moving from raw pixel data to high-level semantic understanding.

Input Stage:

- The model accepts input tensors of shape **(176, 208, 1)**.
- A **Gaussian Noise** layer is immediately applied. This adds slight statistical noise to the inputs during training, acting as a regularizer to improve model robustness and prevent overfitting to clean data.

Early Blocks (Local Feature Extraction):

- **Blocks 1 & 2:** These stages focus on learning low-level features.
- **Attention:** A **CBAM Block** follows the convolution to refine the feature maps immediately.
- **Pooling:** **MaxPooling2D** is used to downsample the spatial dimensions, reducing computational load and providing translation invariance.

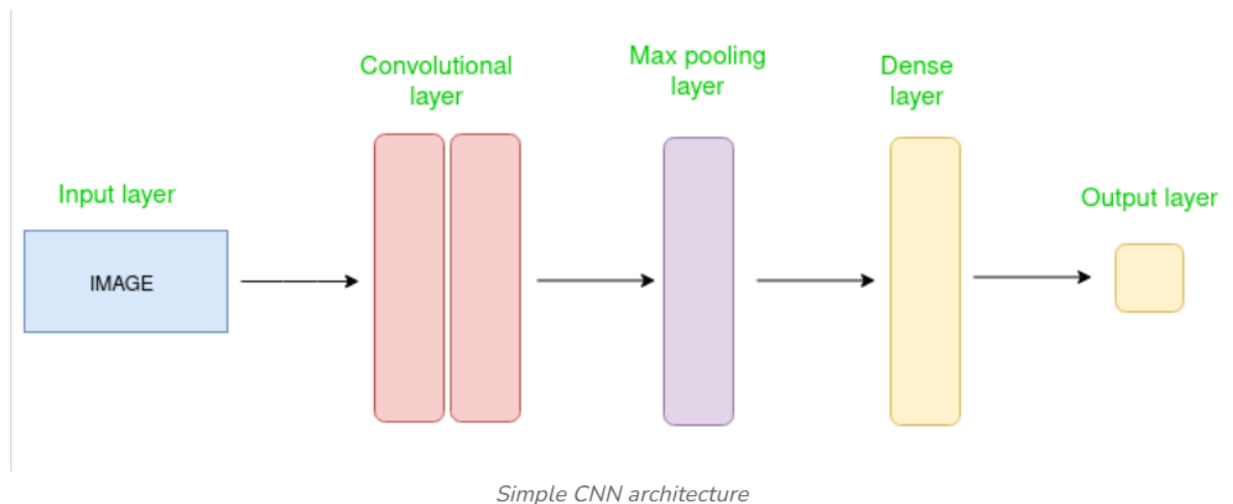
Deep Hybrid Blocks (Global Context Extraction):

- **Blocks 3, 4 & 5:** These stages handle higher-level abstraction using a hybrid approach.
- **Hybrid Branching:** The architecture splits into two parallel conceptual paths:
 - **Path A (CNN):** Continues with standard convolutional layers increasing in depth (64, 128, to 256 filters) to capture complex shapes.
 - **Path B (ViT):** The feature map is flattened into a sequence and passed through a **Vision Transformer Block**. This block utilizes **Multi-Head Attention** to relate different sequence parts (image patches) to one another, followed by a Multi-Layer Perceptron (MLP).
- **Fusion:** The outputs of the CNN path and the ViT path are combined via a **Residual Connection** (addition). This allows the network to retain local detail while integrating global context.

Classification Head:

- **Global Average Pooling:** Converts the 2D feature maps into a 1D vector by averaging each feature map, drastically reducing parameters and minimizing overfitting.
- **Dense Layer:** A fully connected layer with 256 neurons (ReLU activation) interprets the aggregated features.
- **Output Layer:** A final Dense layer with **4 neurons** and **Softmax activation** provides the probability distribution across the four classes (Non, Very Mild, Mild, Moderate).

4.3.1 CNN



Convolutional Neural Network (CNN) is an advanced version of [artificial neural networks \(ANNs\)](#), primarily designed to extract features from grid-like matrix datasets. This is particularly useful for visual datasets such as images or videos, where data patterns play a crucial role. CNNs are widely used in [computer vision](#) applications due to their effectiveness in processing visual data.

CNNs consist of multiple layers like the input layer, Convolutional layer, pooling layer, and fully connected layers. Let's learn more about CNNs in detail.

How Convolutional Layers Works?

Convolution Neural Networks are neural networks that share their parameters.

Imagine you have an image. It can be represented as a cuboid having its length, width (dimension of the image), and height (i.e the channel as images generally have red, green, and blue channels).

Layers Used to Build ConvNets

A complete Convolution Neural Networks architecture is also known as convnets. A convnets is a sequence of layers, and every layer transforms one volume to another through a differentiable function.

Let's take an example by running a convnets on of image of dimension $32 \times 32 \times 3$.

- **Input Layers:** It's the layer in which we give input to our model. In CNN, Generally, the input will be an image or a sequence of images. This layer holds the raw input of the image with width 32, height 32, and depth 3.
- **[Convolutional Layers](#):** This is the layer, which is used to extract the feature from the input dataset. It applies a set of learnable filters known as the kernels to the input images. The filters/kernels are smaller matrices usually 2×2 , 3×3 , or 5×5 shape. it slides over the input image data and computes the dot product between kernel weight and the corresponding input image patch. The output of this layer is referred as feature maps. Suppose we use a total of 12 filters for this layer we'll get an output volume of dimension $32 \times 32 \times 12$.
- **[Activation Layer](#):** By adding an activation function to the output of the preceding layer, activation layers add nonlinearity to the network. it will apply an element-wise activation function to the output of the convolution layer. Some common activation functions are RELU:

$\max(0, x)$, Tanh, Leaky RELU, etc. The volume remains unchanged hence output volume will have dimensions $32 \times 32 \times 12$.

- [Pooling layer](#): This layer is periodically inserted in the convnets and its main function is to reduce the size of volume which makes the computation fast reduces memory and also prevents overfitting. Two common types of pooling layers are max pooling and average pooling. If we use a max pool with 2×2 filters and stride 2, the resultant volume will be of dimension $16 \times 16 \times 12$.

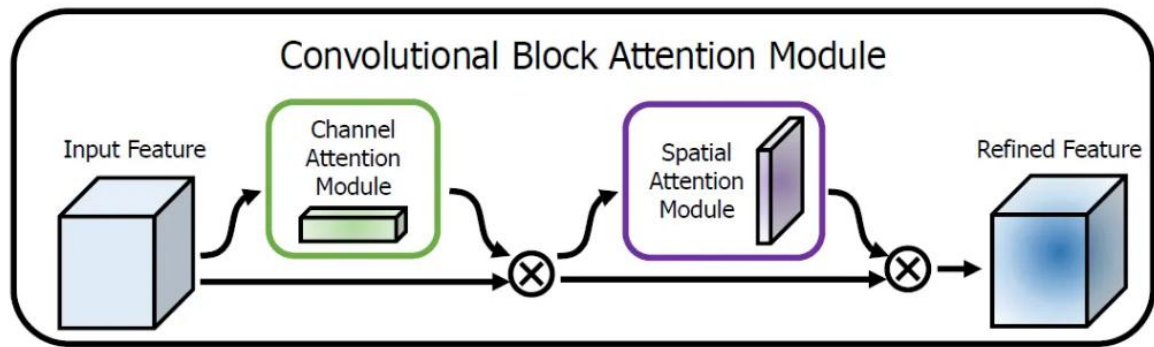
Advantages of CNNs

1. Good at detecting patterns and features in images, videos, and audio signals.
2. Robust to translation, rotation, and scaling invariance.
3. End-to-end training, no need for manual feature extraction.
4. Can handle large amounts of data and achieve high accuracy.

Disadvantages of CNNs

1. Computationally expensive to train and require a lot of memory.
2. Can be prone to overfitting if not enough data or proper regularization is used.
3. Requires large amounts of labeled data.
4. Interpretability is limited, it's hard to understand what the network has learned.

4.4.2 CBAM

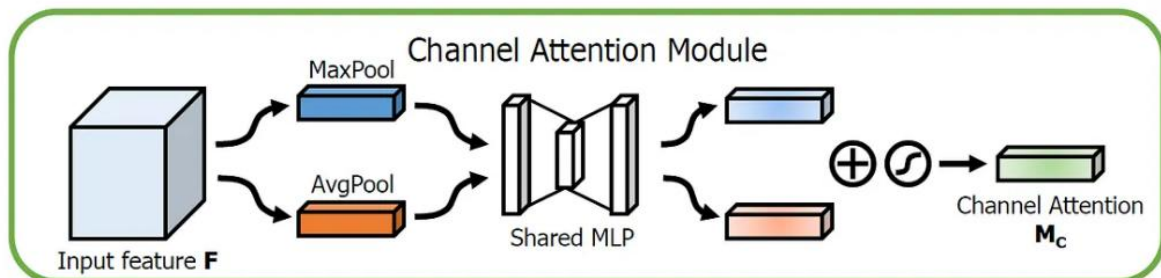


CBAM: General Architecture

- CBAM sequentially infers a **1D channel attention map M_c** with size of **$C \times 1 \times 1$** and a **2D spatial attention map M_s** with the size of **$1 \times H \times W$** :
- where $\otimes$ denotes element-wise multiplication, and F'' is the final refined output.
- Two modules can be placed in a parallel or sequential manner. It is found that the **sequential arrangement gives a better result than a parallel arrangement**.
- For the arrangement of the sequential process, experimental result shows that the **channel-first order is slightly better than the spatial-first**.
- An example of CBAM in a ResBlock is as shown below:

$$F' = M_c(F) \otimes F,$$

$$F'' = M_s(F') \otimes F',$$

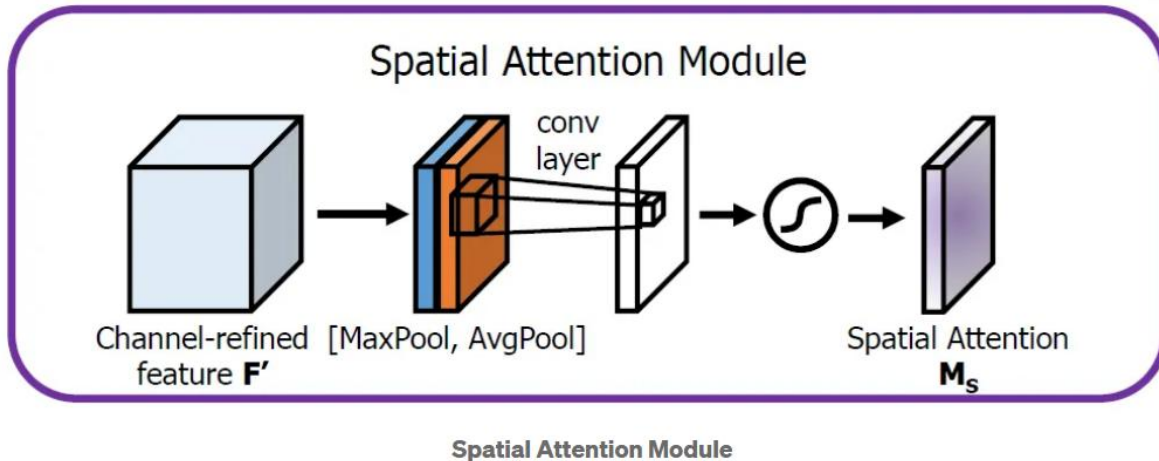


Channel Attention Module

- Channel attention focuses on ‘what’ is meaningful given an input image.
- To compute the channel attention efficiently, **the spatial dimension of the input feature map is squeezed.**
- For aggregating spatial information, **average-pooling has been commonly adopted.** But it is argued that **max-pooling gathers another important clue about distinctive object features** to infer finer channel-wise attention.
- Thus, both average-pooled and max-pooled features are used simultaneously.

$$\begin{aligned} M_c(F) &= \sigma(MLP(AvgPool(F)) + MLP(MaxPool(F))) \\ &= \sigma(W_1(W_0(F_{avg}^c)) + W_1(W_0(F_{max}^c))), \end{aligned}$$

- **F_{cavg} and F_{cmax}** , denote **average-pooled features** and **max-pooled features** respectively. Both descriptors are then forwarded to a shared network to produce our channel attention map M_c .
- The shared network is composed of **multi-layer perceptron (MLP) with one hidden layer.** To reduce parameter overhead, the hidden activation size is set to $R/C=r \times 1 \times 1$, where r is the reduction ratio.
- After the shared network is applied to each descriptor, **the output feature vectors are merged using element-wise summation.**
- σ denotes the sigmoid function. This $M_c(F)$ is element-wise multiplied with F to form F' .



- The **spatial attention focuses on ‘where’ is an informative part**, which is complementary to the channel attention.
- To compute the spatial attention, average-pooling and max-pooling operations are applied along the channel axis and then concatenate them to generate an efficient feature descriptor.
- A convolution layer is then applied to generate a spatial attention map $M_s(F)$ with the size of $R \times H \times W$ which encodes where to emphasize or suppress.

$$\begin{aligned}
 \mathbf{M}_s(\mathbf{F}) &= \sigma(f^{7 \times 7}([\text{AvgPool}(\mathbf{F}); \text{MaxPool}(\mathbf{F})])) \\
 &= \sigma(f^{7 \times 7}([\mathbf{F}_{\text{avg}}^s; \mathbf{F}_{\text{max}}^s])),
 \end{aligned}$$

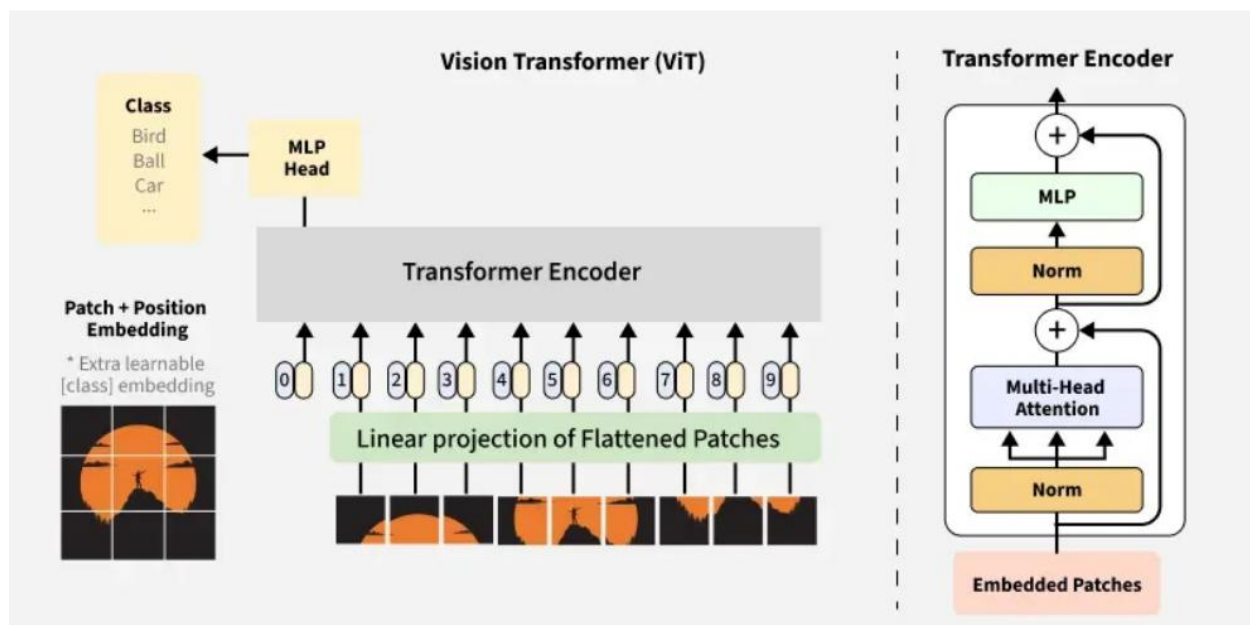
- Specifically, **aggregate channel information of a feature map by using two pooling operations, generating two 2D maps: $\mathbf{F}_{\text{avg}}^s$ with the size of $1 \times H \times W$ and $\mathbf{F}_{\text{max}}^s$ with the size of $1 \times H \times W$.**
- σ denotes the sigmoid function and **$f^{7 \times 7}$ represents a convolution operation with the filter size of 7×7 .**

4.4.3. Vision Transformer

The Vision Transformer builds upon the transformer architecture initially introduced by Vaswani et al. in 2017 for NLP tasks. Transformers are highly effective at processing sequential data, utilizing self-attention to model dependencies between different parts of the input. Vision Transformers apply this architecture to image data, treating the image as a sequence of patches instead of a grid of pixels.

Vision Transformer architecture comprises several key stages:

1. Image Patching and Embedding
2. Positional Encoding
3. Transformer Encoder
4. Classification Head (MLP Head)



Architecture and Working of Vision Transformer (ViT)

1. Image Patching and Embedding

The first and most critical step in the ViT pipeline is to convert the image into a sequence of patches, similar to the tokens in an NLP model.

- **Patch Splitting:** The input image, usually of size $H \times W \times C$ (height, width, and channels), is divided into fixed-size patches. For

example, an image of size 224x224 can be split into non-overlapping 16x16 patches, resulting in $\frac{224}{16} \times \frac{224}{16} = 14 \times 14 = 196$ patches.

- **Patch Flattening:** Each patch is then flattened into a 1D vector. A patch of size $P \times P \times C$ (e.g., 16x16x3) is reshaped into a vector of size $P^2 \times C$, creating 196 patch vectors for an image.
- **Patch Embedding:** Each flattened patch is projected into a higher-dimensional space (embedding dimension D) through a **learnable linear projection**. This linear transformation enables the model to learn richer feature representations for each patch. The result is a sequence of patch embeddings, each representing a part of the image. The total number of patches in the sequence is $N = \frac{H}{P} \times \frac{W}{P}$, where N is the number of patches. For instance, with a 224x224 image and 16x16 patches, we have 196 patches.

2. Positional Encoding

Transformers do not inherently capture the spatial order of input sequences. Since the patches are processed as independent tokens, it's essential to introduce **positional encodings** to retain the spatial structure of the original image.

- **Positional Embedding:** Positional encodings are added to each patch embedding to encode information about the location of patches within the image. These embeddings help the model understand the spatial relationships between patches, similar to how transformers in NLP encode the positions of words in a sentence.
- **Learned vs. Fixed Positional Encoding:** In ViTs, positional encodings can either be learned during training or predefined (fixed). Most implementations of Vision Transformers use learnable positional encodings.

3. Transformer Encoder Layers

Once the patches are embedded and augmented with positional information, they are passed through a stack of **transformer encoder layers**. These layers consist of two primary components: **Multi-Head Self-Attention (MSA)** and a **Feed-Forward Neural Network (FFN)**.

1. Multi-Head Self-Attention (MSA)

- **Self-Attention:** The self-attention mechanism allows each patch to attend to every other patch in the sequence. This means that the transformer can model long-range dependencies and relationships between different parts of the image. Each patch computes a weighted sum of the values of all other patches based on its similarity to them, known as the attention score.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Where Q (query), K (key), and V (value) are learned linear projections of the input patch embeddings.

The dot product between queries and keys determines the attention score, and softmax normalizes it.

The weighted sum of values determines the output.

- **Multi-Head Attention:** The attention mechanism is computed in parallel across multiple attention heads, allowing the model to focus on different parts of the image simultaneously.

2. Feed-Forward Network (FFN)

After self-attention, the patches are passed through a **feed-forward network (FFN)**. The FFN consists of two fully connected layers with a non-linear activation function (typically GELU) in between.

Each transformer encoder layer includes **residual (skip) connections** and **layer normalization** to stabilize training and improve convergence. These techniques ensure that the deeper layers do not lose important information from the earlier layers.

Stacking Encoder Layers

Multiple transformer encoder layers (e.g., 12, 24 layers) are stacked on top of each other. Each layer refines the patch embeddings, allowing the model to build more complex and abstract representations of the image.

4. Classification Token (CLS Token)

In Vision Transformers, a special **classification token (CLS token)** is introduced at the beginning of the input sequence. This token serves a critical role: it gathers information from all the patches throughout the transformer layers.

The CLS token learns to represent the entire image by attending to the different patches through the self-attention mechanism. At the output of the transformer layers, the CLS token is extracted and passed to a classifier for the final prediction.

5. MLP Head (Classification Head)

After the transformer encoders process the sequence of patches and the CLS token, the output corresponding to the CLS token is used for classification.

The output of the CLS token is fed into an MLP, typically consisting of one or two fully connected layers. A softmax layer is applied at the end of the MLP for classification tasks, predicting the image's label.

Vision Transformer Architecture Summary

To summarize, the Vision Transformer architecture involves the following key steps:

1. **Input Image Processing:** The input image is divided into patches, which are flattened and embedded using a linear projection.
2. **Positional Encoding:** Positional encodings are added to the patch embeddings to retain spatial information.
3. **Transformer Encoder:** The patch embeddings (along with the CLS token) are passed through multiple transformer encoder layers, which include multi-head self-attention and feed-forward networks.
4. **Classification Head:** The CLS token's output is extracted and fed into an MLP for final classification.

Vision Transformer (ViT) vs. Convolutional Neural Networks (CNNs)

- **Local vs. Global Attention:** CNNs capture local spatial features through convolutions, whereas ViTs capture global relationships using self-attention.
- **Inductive Biases:** CNNs have built-in inductive biases, such as locality and translation invariance, which make them effective at image tasks with smaller datasets. Vision Transformers, on the

other hand, rely on data to learn these patterns, making them more flexible but data-hungry.

- **Training Data Requirements:** Vision Transformers typically require large amounts of training data to achieve their best performance, while CNNs can perform well even with smaller datasets.

Strengths of Vision Transformer Architecture

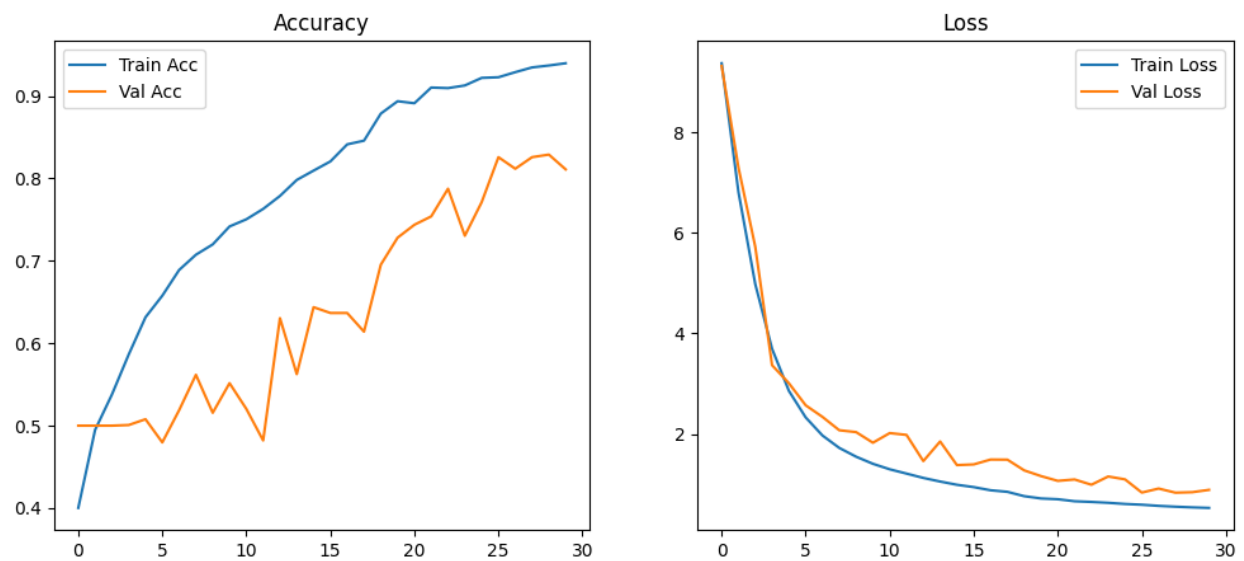
- **Global Context:** ViTs are excellent at capturing long-range dependencies between image patches, giving them a better understanding of the global context of images.
- **Scalability:** ViTs scale well with larger datasets and deeper architectures, making them highly effective for large-scale vision tasks.
- **Flexibility:** Since ViTs don't rely on convolutions, they can be easily adapted to different tasks and domains beyond image classification, such as video analysis and object detection.

Challenges and Limitations

- **Data Requirements:** ViTs are data-hungry and typically require large-scale datasets for effective training. Fine-tuning ViTs pre-trained on large datasets (like ImageNet) is a common practice when data is limited.
- **Computational Resources:** Vision Transformers demand significant computational power, especially during training, due to the quadratic complexity of the self-attention mechanism.

5. Results

- **Learning Curve:**



- **Model evaluation**

Class	Precisi on	Recall	F1	Count
Mild Demented	0.84	0.75	0.79	179
Moderate Demented	1.00	0.77	0.87	13
Non Demented	0.87	0.88	0.87	640

Class	Precisi on	Recall	F1	Count
Very Mild Demented	0.76	0.79	0.78	448
Accuracy			0.83	1280
Macro Avg	0.87	0.80	0.83	1280
Weighted Avg	0.83	0.83	0.83	1280

6. Implementation details

The implementation was carried out using the **TensorFlow/Keras** framework, leveraging its robust ecosystem for building complex custom layers.

1. **Optimizer:** The **Adam** optimizer was selected for its adaptive learning rate capabilities, set to an initial learning rate of **0.0001**. This relatively low learning rate ensures stable convergence for the hybrid architecture.
2. **Regularization:** Given the limited size of medical datasets, overfitting is a primary concern. Extensive regularization techniques were employed:
 - a. **L2 Regularization:** A weight decay factor of **0.005** was applied to kernel weights to penalize large weights and encourage simpler models.
 - b. **Dropout:** Dropout rates ranging from **0.3 to 0.5** were strategically placed after dense and transformer blocks to

randomly deactivate neurons during training, forcing the network to learn redundant and robust features.

3. **Callbacks:** Two key callbacks were used to optimize the training process:

- a. **ReduceLROnPlateau:** Automatically halves the learning rate if the validation loss stagnates for 3 epochs, allowing the model to fine-tune its weights when it hits a plateau.
- b. **EarlyStopping:** Terminates training if validation accuracy does not improve for 8 consecutive epochs, saving computational resources and preventing overfitting.

4. **Hyperparameters:** A **Batch Size of 16** was chosen. This smaller batch size was empirically tuned to provide the best balance between training stability and memory usage for the attention-heavy architecture.

7. Conclusion

This project successfully demonstrates the efficacy of a Hybrid CNN-ViT model for the nuanced task of Alzheimer's stage classification. By addressing the critical issue of class imbalance through oversampling and integrating CBAM and Vision Transformers, the model achieves a high level of robustness and accuracy. This architecture not only identifies severe cases with high precision but also shows promise in distinguishing the challenging early stages of the disease, validating the potential of hybrid deep learning models in computer-aided medical diagnosis.

References

1. <https://www.geeksforgeeks.org/deep-learning/vision-transformer-vit-architecture/>
2. Woo et al. (2018), "CBAM: Convolution Block Attention Module "
3. Dosovitskiy et al. (2020), "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale"
4. Al-Amin et al. (2025), "Hybrid-RViT: Hybridizing ResNet-50 and Vision Transformer"